2. Requirements linkage (for the presentation)

This is based on the code and the live checks from this session. Statuses are what I can defend today.

Req	What the code does	Where to point	Evidence	Status
FR-001 Submit	Factory validates category and fields, status New, severity Low by default	services/RequestFactory.js, services/requestTypes/, middleware/request.controller.js	10 factory tests, live POST returns 201	Partial (no UI-to-DB check in the full flow)
FR-002 Status	Transition table, version check, audit row written in the same transaction	services/RequestService.js (updateRequestStatus)	Live: invalid transition 400, stale version 409, history recorded	Met (backend)
FR-003 Own requests and history	Resident queries scoped to their own ID; history returned on detail	services/RequestService.js (getRequests, getRequestById)	Live: other resident gets 404, own list excludes theirs	Met (backend); UI display pending
FR-004 Notifications	Observer exists but only logs	services/observers/ResidentNotificationObserver.js	None	Not met
FR-005 Filter	Parameterised filters,	RequestService.getRequests, db/002-request-filter-indexes.sql	Live: category, severity	Met (backend);

	whitelisted sorting		and status filters return no mismatches	queue UI pending
FR-006 Assign	assigned_to column exists, no operation	db/001-initial_Schema.sql	None	Not met
FR-007 Comments	Nothing	None	None	Not met
FR-008 Dashboard counts	Frontend charts compute totals; backend observer is a placeholder	frontend/.../StaffDashboard.jsx, DashboardCountsObserver.js	Visual only	Partial
FR-009 Grouping	Charts group by status, severity, category	StaffDashboard.jsx	Visual only	Partial
FR-010 Reference number	Generated on create, unique in the database	RequestService.createRequest, schema UNIQUE (reference_number)	Live: two submissions, distinct references	Met
FR-011 Severity	Stored, validated, default Low; High	RequestFactory.js, queue and dashboard	Factory tests; UI highlight visual	Partial (visual)

	shown in orange			
NFR-001 Access control	JWT verified on each request, role checks per route, scrypt password hashing	middleware/auth.middleware.js, routes/auth.routes.js, auth/password.js	Live: 401, 403 on tamper and role misuse	Partial (no login rate limiting)
NFR-002 High visible	Same as FR-011 visual	Queue and dashboard	Visual	Partial
NFR-003 Usability (3 minutes)	Not measured	None	None	Not met
NFR-004 History never overwritten	History is a separate table written in the same transaction; stale update leaves it unchanged	RequestService.updateRequestStatus, observers/AuditHistoryObserver.js	Live: content hash identical after a rejected update	Met (backend)
NFR-005 List loads in 3 seconds	Indexes on filter columns	db/002-request-filter-indexes.sql	Historical API average only, no browser timing	Not met (not yet measured)

NFR-006 Complex association	complex_id on all records, not null	Schema	0 null rows	Partial (no complexes table)

Presentation tip: pick three to walk through live, for example FR-002 (transition, 400, 409 and history), NFR-004 (hash before and after), and FR-003 (cross-resident 404). Those are the strongest proofs you have.

3. Showing the layered monolith

The layers you can show:

- **API (request handling):** middleware/request.controller.js and routes/auth.routes.js receive HTTP requests. The controller calls RequestService.
- **Service (business rules):** services/RequestService.js, RequestFactory.js, and the observers. These hold the rules: transitions, validation, scoping, audit.
- **Persistence (database):** the SQL in db/ and the connection in config/db.js.

Two checks you can run during the presentation:

powershell

Select-String -Path src\services*.js -Pattern "\breq\.|\bres\."

This should print nothing. The service layer doesn't know about HTTP.

powershell

Select-String -Path src\middleware\request.controller.js -Pattern "RequestService"

This shows the controller calls the service and nothing else.

Be honest about these gaps, because a lecturer checking the repo will find them:

- RequestService.js runs SQL directly. The transaction code is in the service, not in a separate persistence layer.
- routes/auth.routes.js runs SQL directly, bypassing the service layer.
- There's no src/persistence folder. ADR-ARCH-001 names that folder, so the repo doesn't match the decision as written.

- The controller is in middleware/ rather than api/, which is misleading. routes/request.routes.js is unused, and it's written in CommonJS.

What to say: "The monolith and the layering rules are implemented, with two documented deviations. The service layer currently owns persistence, and the auth routes bypass the service layer. Both are recorded as technical debt, with a planned move to src/persistence." That's accurate, and it's what the M4 brief rewards: honest deviation with a stated consequence.

If you have time before the demo, the fix is to move the SQL in RequestService.js into src/persistence/requestRepository.js and have the service call it. That's about an hour of careful work and needs testing afterwards. I wouldn't do it in the last day before a freeze unless you're sure. Otherwise, correct the PED: update the architecture section and the folder diagram so they match the repo, and list the two deviations in the technical debt register.